

CRIS에서 다양한 스펙으로 프로그램의 원하는 성질 검증하기

2026.7.28 – 7.30 프로그램 검증 워크숍
박선호

Intro

Q: 무슨 성질을 검증하고 싶으신가요?

다를 주제



1. 프로그램이 안 터지는지

```
int a[1];  
a[1] = 0; // out-of-bound  
  
scanf("%d%d", &x, &y);  
z = x/y; // div-by-zero  
  
int x = (...);  
assert(x != 0); // failed
```

2. 알고리즘이 올바른 값을 계산하는지

LIFO (Last-In-First-Out)

```
Stack s;  
s.push(0); s.push(1);  
s.pop(); // 1  
s.pop(); // 0
```

3. 기타 더 복잡한 성질

- Type soundness
- 컴파일러 correctness
- Security
- Deadlock-freedom
- Fairness/Liveness
- ...

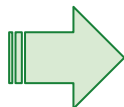


gil.hur@sf
.snu.ac.kr

Running Example

```
int main(void) {  
    int x, y, a, b;  
    My_stack *s = new My_stack();  
  
    scanf("%d%d", &x, &y);  
    s->push(x);      s->push(y);  
    a = s->pop();     b = s->pop();  
  
    assert((a, b) == (y, x)); // LIFO  
    printf("%d%d", a, b);  
    return 0;  
}
```

목표1: 이 프로그램이 안 터진다는 사실만 보이고 싶어요!
(no memory error, no assert failure)



목표2: 이런 구체적인 예시 말고,
일반적으로 My_stack이 LIFO를
지킨다는 사실을 보이고 싶어요!



핵심 문제: 어떤 **스펙**을 써야 이 목표를 달성 가능한가?
(두 목표 공통 문제)

목표1: 이 프로그램이 안 터진다는 사실만 보이고 싶어요!

Running Example을 검증하기 위한 고찰



Recall: CRIS는 **refinement** 를 검증하는 framework 이다

구현 코드 $\models_{\text{beh}}$ 스펙

의미: 구현 코드에서 나올 수 있는 모든 행동은
스펙에도 있어야 한다.

Contrapositive (!):

스펙에서 나올 수 없는 행동은 구현에도 없어야 한다.

⇒ 아이디어: 에러가 절대 발생 안하는 스펙 코드를 생각하자

목표1: 이 프로그램이 안 터진다는 사실만 보이고 싶어요!

Running Example을 검증하기 위한 스펙

Bug-free simple spec

```
int main(void) {  
    int x, y, a, b;  
    My_stack *s = new My_stack();  
  
    scanf("%d%d", &x, &y);  
    s->push(x);      s->push(y);  
    a = s->pop();     b = s->pop();  
  
    assert((a, b) == (y, x)); // LIFO  
    printf("%d%d", a, b);  
    return 0;  
}
```

$\sqsubseteq_{\text{beh}}$

```
int main(void) {  
    int x, y, a, b;  
  
    scanf("%d%d", &x, &y);  
  
    a = y; b = x;  
  
    printf("%d%d", a, b);  
    return 0;  
}
```

목표1: 이 프로그램이 안 터진다는 사실만 보이고 싶어요!

Running Example을 검증하기 위한 스펙

(Advanced) Bug-free simpler spec

```
int main(void) {  
    int x, y, a, b;  
    My_stack *s = new My_stack();
```

```
    scanf("%d%d", &x, &y);  
    s->push(x);      s->push(y);  
    a = s->pop();    b = s->pop();
```

```
    assert((a, b) ==  
    printf("%d%d",  
    return 0;
```

```
}
```

$\sqsubseteq_{\text{beh}}$

```
int main(void) {  
    int x, y, a, b;  
  
    scanf("%d%d", &x, &y);  
    // LIFO 조차도 검증하기 싫을 때  
    a = rand(); b = rand();
```

남은 할 일

1. 코드를 CRIS 위로 옮기기
2. Final theorem (refinement) 쓰기
3. AI 돌리기

```
    printf("%d%d", a, b);  
    return 0;
```

목표1: 이 프로그램이 안 터진다는 사실만 보이고 싶어요!

Running Example을 CRIS module로 작성하기

main.c

```
int main(void) {  
    int x, y, a, b;  
    My_stack *s = new My_stack();  
  
    scanf("%d%d", &x, &y);  
    s->push(x);      s->push(y);  
    a = s->pop();     b = s->pop();  
  
    assert((a, b) == (y, x)); // LIFO  
    printf("%d%d", a, b);  
    return 0;  
}
```

```
void My_stack::push(int val) {  
    (blahblah)  
}
```

my_stack.c

CRIS에서의 main 모듈

```
main_sig := fnsig "main"  
          (input type) (return type)  
  
main_def : itree := (... 다음 슬라이드 ...)  
  
main_mod : SMod.t := {  
    (다른 field는 생략)  
    fnsems := { main_sig # main_def }  
}
```

```
new_stack_sig # ..._def,  
push_sig # push_def,  
pop_sig # pop_def }
```

CRIS에서의 my_stack 모듈

목표1: 이 프로그램이 안 터진다는 사실만 보이고 싶어요!

Running Example을 CRIS itree로 옮기기

- 어제 (화요일) 설명한 interaction tree (itree) 와 같지만, 다양한 notation을 통해 사실상 프로그래밍하는 느낌과 동일함

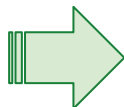
(pseudo-) syntax of itree (in CRIS) — 많이 생략됨. 곧 더 많은 내용 설명함

(itree) E ::=	Ret v	: “v”가 return value
	E ;; E	: Sequencing
v: 임의의 Rocq 값 x: variable	x ← E ;; E	: 앞의 E를 실행한 결과를 x라 하고, 뒤의 E 실행
P: 임의의 Rocq proposition	ccallU (fnsig) (arg)	: function call (해당 fnsig의 정의가 적힌 모듈과 linking되어야 정상실행)
	trigger (event)	: 이벤트 발생 (예시: I/O event)
	assume P	: assert 구문에 해당 (P가 거짓이면 error (UB)) 8

목표1: 이 프로그램이 안 터진다는 사실만 보이고 싶어요!

Running Example을 CRIS itree로 옮기기 — Main 모듈

```
int main(void) {  
    int x, y, a, b;  
    My_stack *s = new My_stack();  
  
    scanf("%d%d", &x, &y);  
    s->push(x);      s->push(y);  
    a = s->pop();     b = s->pop();  
  
    assert((a, b) == (y, x)); // LIFO  
    printf("%d%d", a, b);  
    return 0;  
}
```



```
def main_def : itree ≡  
    s ← ccallU new_stack_sig () ;;  
  
    (x, y) ← trigger (IO "scanf") ;;  
    ccallU push_sig (s, x) ;;;  
    ccallU push_sig (s, y) ;;;  
    a ← ccallU pop_sig (s) ;;  
    b ← ccallU pop_sig (s) ;;  
  
    assume((a,b) = (y,x)) ;;;  
    trigger (IO "printf" (a, b)) ;;;  
    Ret 0
```

목표1: 이 프로그램이 안 터진다는 사실만 보이고 싶어요!

Running Example을 CRIS itree로 옮기기 — Stack 모듈

```
def new_stack_def : itree ≡  
  s ← ccallU malloc_sig () ;;  
  ccallU store_sig (s, NULL) ;;  
  Ret s
```

```
def pop_def (s: Loc) : itree ≡  
  head ← ccallU load_sig (s) ;;  
  if (head = NULL)  
  then Ret ()  
  else (next, val) ← (...) ;; (...)
```

참고사항: CRIS는 **language-generic**하다.

⇒ 구체적인 메모리 alloc/load/store/free 에 대한 predefined rule이 없음

⇒ 즉, 메모리 access에 대한 operational semantics를 **메모리 모듈**로써 따로 정의해 줘야 함



목표1: 이 프로그램이 안 터진다는 사실만 보이고 싶어요!

Running Example을 CRIS itree로 옮기기 — 메모리 모듈

$$\frac{\ell \in \text{dom}(M)}{\langle x := \text{alloc}(), \Sigma, M \rangle \Rightarrow \langle \text{skip}, \Sigma[x := \ell], M[\ell := \text{skull}] \rangle}$$

할당: 새로운 ℓ 을 찾아서 맵 만들기

$$\frac{\Sigma[x] = \ell \quad M[\ell] = v}{\langle y := \text{load}(x), \Sigma, M \rangle \Rightarrow \langle \text{skip}, \Sigma[y := v], M \rangle}$$

읽기: ℓ 의 값을 M 으로부터 얻기

$$\frac{\Sigma[x] = \ell \quad \ell \in \text{dom}(M)}{\langle \text{store}(x, w), \Sigma, M \rangle \Rightarrow \langle \text{skip}, \Sigma, M[\ell := w] \rangle}$$

쓰기: ℓ 의 새 값을 M 에 쓰기

$$\frac{\Sigma[x] = \ell \quad M[\ell] = v}{\langle \text{free}(x), \Sigma, M \rangle \Rightarrow \langle \text{skip}, \Sigma, M \setminus \{\ell := v\} \rangle}$$

해제: ℓ 의 정보를 M 에서 지우기

memory state

- $M \in \text{Loc} \rightarrow \text{Val}$

목표1: 이 프로그램이 안 터진다는 사실만 보이고 싶어요!

Running Example을 CRIS itree로 옮기기 — 메모리 모듈

$\ell \notin \text{dom}(M)$

$\langle x := \text{alloc}(), \Sigma, M \rangle \Rightarrow \langle \text{skip}, \Sigma[x := \ell], M[\ell := \text{skull}] \rangle$

def alloc(): itree $\equiv$

$M \leftarrow \text{cgetU}();;$

$\ell \leftarrow \text{choose}(\text{Loc} \setminus \text{dom}(M));;$

$\text{cputU}(M[\ell := \text{skull}]);;$

Ret ℓ

[모듈-로컬 상태 접근하기]

cgetU: 상태 읽기

cputU: 상태 쓰기

$\Sigma[x] = \ell$

$M[\ell] = v$

$\langle y := \text{load}(x), \Sigma, M \rangle \Rightarrow \langle \text{skip}, \Sigma[y := v], M \rangle$

def load(ℓ): itree $\equiv$

$M \leftarrow \text{cgetU}();;$

assume($\ell \in \text{dom}(M)$);;

Ret $M[\ell]$

목표1: 이 프로그램이 안 터진다는 사실만 보이고 싶어요!

마지막 Step: Final theorem 서술하기

```
int main(void) {  
    int x, y, a, b;  
    My_stack *s = new My_stack();  
  
    scanf("%d%d", &x, &y);  
    s->push(x); s->push(y);  
    a = s->pop(); b = s->pop();  
  
    assert((a, b) == (y, x)); // LIFO  
    printf("%d%d", a, b);  
    return 0;  
}
```

$\sqsubseteq_{\text{beh}}$

```
int main(void) {  
    int x, y, a, b;  
  
    scanf("%d%d", &x, &y);  
    a = y; b = x;  
  
    printf("%d%d", a, b);  
    return 0;  
}
```



Main 모듈 ★

Stack 모듈 ★

메모리 모듈

$\sqsubseteq_{\text{beh}}$

간단한 Main 모듈

day3/prime/PrimeAll.v

Theorem behavioral_refinement :

```
∃ β τ  
  (Hinv : invGS Γ Σ α)  
  (_ : crisG Γ Σ α β τ _ Hinv)  
  (_ : llistGS)  
  (_ : memGS)  
  src_res tgt_res,  
  refines_lmod  
    (Mod.to_lmod mod_tgt tgt_res)  
    (Mod.to_lmod mod_top src_res).
```

목표2: 이런 구체적인 예시 말고, 일반적으로 My_stack이 LIFO를 지킨다는 사실을 보이고 싶어요!

자료구조 스펙을 작성하는 방법 — try

자료구조에 대해 우리가 가지고 있는 지식 (구현에 상관 X)

- `s = new_stack()`: `s`에 비어 있는 새 스택이 만들어져요
- `push(s, v)`: `s`에 있는 스택에 `v`라는 값을 넣어요
- `pop(s)`: `s`에 있는 스택에 맨 마지막에 넣어진 값을 빼요
- `q = new_queue()`: `q`에 비어 있는 새 큐가 만들어져요
- `enq(q, v)`: `q`에 있는 큐에 `v`라는 값을 넣어요
- `deq(q)`: `q`에 있는 큐에 있는 값 중 맨 먼저 넣어진 값을 빼요
- `m = new_map()`: `m`에 비어 있는 새 맵이 만들어져요
- `insert(m, k, v)`: `m`에 있는 맵에 (`k:=v`) 라는 key-value pair를 넣어요
- `get(m, k)`: `m`에 있는 맵에 키 값 `k`에 대응되는 value를 알려줘요

아이디어: 자료구조의
수학적 상태 (무슨 값이
있는지) 만으로 스펙을
작성하자

목표2: 이런 구체적인 예시 말고, 일반적으로 `My_stack`이 LIFO를 지킨다는 사실을 보이고 싶어요!

자료구조 스펙을 작성하는 방법 — try

- `s = new_stack()`: `s`에 비어 있는 새 스택이 만들어져요
- `push(s, v)`: `s`에 있는 스택에 `v`라는 값을 넣어요
- `pop(s)`: `s`에 있는 스택에 맨 마지막에 넣어진 값을 빼요

Definition **IsStack**: $\text{MemState} \rightarrow \text{Loc} \rightarrow \text{list (Val)} \rightarrow \text{Prop}$

IsStack (M, s, L) 해석: 현재 메모리 상태 M 에서 스택 s 에는 L 의 값들이 있다.

$M_1 \rightarrow s = \text{new_stack}() \rightarrow M_2 \quad \text{IsStack (M}_2, s, [])$

$M_1 \longrightarrow \text{push}(s, v) \longrightarrow M_2 \quad \text{IsStack (M}_1, s, L) \wedge \text{IsStack (M}_2, s, v :: L)$

$M_1 \longrightarrow \text{pop}(s) \longrightarrow M_2 \quad \text{IsStack (M}_1, s, v :: L) \wedge \text{IsStack (M}_2, s, L)$
성공할 경우

목표2: 이런 구체적인 예시 말고, 일반적으로 `My_stack`이 LIFO를 지킨다는 사실을 보이고 싶어요!

자료구조 스펙을 작성하는 방법 — `try`의 문제점

$M_1 \rightarrow s = \text{new_stack}() \rightarrow M_2 \quad \text{IsStack}(M_2, s, [])$

$M_1 \longrightarrow \text{push}(s, v) \longrightarrow M_2 \quad \text{IsStack}(M_1, s, L) \wedge \text{IsStack}(M_2, s, v :: L)$

$M_1 \longrightarrow \underset{\text{성공할 경우}}{\text{pop}(s)} \longrightarrow M_2 \quad \text{IsStack}(M_1, s, v :: L) \wedge \text{IsStack}(M_2, s, L)$

Q: 만약 M_2 에서 메모리 쓰기 (e.g., `ℓ.store(1)`) 가 일어나서 M_3 로 바뀌면?

⇒ `IsStack`이 M_3 에서도 여전히 성립한다는 것을 보이기가 **까다롭다!**

(**Interference** problem: 이 `ℓ`이 만약 스택이 내부적으로 할당한 메모리였다면?)

목표2: 이런 구체적인 예시 말고, 일반적으로 `My_stack`이 LIFO를 지킨다는 사실을 보이고 싶어요!

Separation logic을 이용하여 스택 작성하기

- 더이상 `IsStack` 처럼 전체 메모리 상태에 대해 수학적 서술을 하지 않는다.
- 대신 ownership 기반의 logical resource를 사용한다.

Definition **IsStack**: $\text{MemState} \rightarrow \text{Loc} \rightarrow \text{list (Val)} \rightarrow \text{Prop}$

`IsStack (M, s, L)` 해석: 현재 메모리 상태 `M`에서 스택 `s`에는 `L`의 값들이 있다.



Definition **IsStack**: $\text{Loc} \rightarrow \text{list (Val)} \rightarrow \text{iProp } \Sigma$

`IsStack (s, L)` 해석:

- 현재 스택 `s`에는 `L`의 값들이 있다.
- 스택 `s`가 접근 가능한 메모리 영역의 **ownership**을 가지고 있다. (남들은 접근 불가) 17

목표2: 이런 구체적인 예시 말고, 일반적으로 My_stack이 LIFO를 지킨다는 사실을 보이고 싶어요!

Separation logic을 이용하여 스펙 작성하기

(pseudo-) syntax of itree (in CRIS)

(itree) $E ::=$

- $\text{Ret } v$
- $| E ;; E$
- $| x \leftarrow E ;;; E$
- $| (\text{생략})$

v : 임의의 Rocq 값
 x : variable
 R : Logical 리소스

$ \text{take (Type)}$	: 지정된 type에 속하는 값을 아무거나 하나 받음
$ \text{Assume } R$	: R 의 ownership을 context에서 받음
$ \text{choose (Type)}$	: 지정된 type에 속하는 값을 아무거나 하나 고름
$ \text{Guarantee } R$	: R 의 ownership을 context에 돌려줌

itree에 리소스가 있을 경우 실행 모델:

- 기존: $(\text{itree}, \text{context}) \rightarrow (\text{itree}, \text{context})$
- 현재: $(\text{itree}, \text{context}, \text{set of resources}) \rightarrow (\text{itree}, \text{context}, \text{set of resources})$

목표2: 이런 구체적인 예시 말고, 일반적으로 My_stack이 LIFO를 지킨다는 사실을 보이고 싶어요!

Separation logic을 이용하여 스택 작성하기

```
def new_stack_def : itree ≡  
  s ← choose (Loc) ;;  
  Guarantee (Stack(s, [])) ;;  
  Ret s
```

```
def push_def (s: Loc, v: Val) : itree ≡  
  L ← take (list (Val)) ;;  
  Assume (Stack(s, L)) ;;  
  Guarantee (Stack(s, v :: L)) ;;  
  Ret ()
```

```
def pop_def (s: Loc) : itree ≡  
  L ← take (list (Val)) ;;  
  Assume (Stack(s, L)) ;;  
  match L with  
  | [] ⇒  
    Guarantee (Stack(s, [])) ;; Ret ()  
  | v :: L' ⇒  
    Guarantee (Stack(s, L')) ;; Ret v  
  end
```

목표2: 이런 구체적인 예시 말고, 일반적으로 My_stack이 LIFO를 지킨다는 사실을 보이고 싶어요!

Separation logic을 이용하여 스펙 작성하기

- 스펙의 이해를 돕기 위한 client 예제

Callee-side

```
def main_def : itree ≡
```

```
  s ← ccallU new_stack_sig () ;;
```

```
  Guarantee (Stack(s, [])) ;;
```

```
  ccallU push_sig (s, 0) ;;
```

```
  Assume (Stack(s, [0])) ;;
```

```
  Guarantee (Stack(s, [0])) ;;
```

```
  v ← ccallU pop_sig (s) ;;
```

```
  Assume (Stack(s, [0])) ;;
```

```
  Guarantee (Stack(s, [])) ;;
```

```
  assume (v = 0) ;;
```

```
  Ret 0
```

목표2: 이런 구체적인 예시 말고, 일반적으로 My_stack이 LIFO를 지킨다는 사실을 보이고 싶어요!

마지막 Step: Final theorem 서술하기

Stack 모듈 (구현)

```
def new_stack_def : itree ≡  
  s ← ccallU malloc_sig () ;;  
  ccallU store_sig (s, NULL) ;;  
  Ret s  
  
def pop_def (s: Loc) : itree ≡  
  head ← ccallU load_sig (s) ;;  
  if (head = NULL)  
  then Ret ()  
  else (next, val) ← (...) ;; (...)
```



메모리 모듈

$\sqsubseteq_{\text{beh}}$

Stack 모듈 (스펙)

```
def new_stack_def : itree ≡  
  s ← choose (Loc) ;;  
  Guarantee (Stack(s, [])) ;;  
  Ret s  
  
def pop_def (s: Loc) : itree ≡  
  L ← take (list (Val)) ;;  
  Assume (Stack(s, L)) ;;  
  match L with  
  | [] ⇒  
    Guarantee (Stack(s, [])) ;; Ret ()  
  | v :: L' ⇒  
    Guarantee (Stack(s, L')) ;; Ret v  
  end
```

감사합니다!

못다한 이야기:

- 실제로 logical resource는 어떻게 정의를 주고, 어떤 invariant를 통해 증명하나?
 - 뒤에 오는 슬라이드 참고
- Logical resource의 고급 기능
 - Monotone하게 변하는 성질에 대해서 과거 상태의 snapshot을 기억하는 기능
 - Fractional ownership: ownership을 여럿이서 나누어 가지는 기능

위 내용은 슬라이드(이 다음 페이지부터)와 오늘 튜토리얼 예제로 있습니다!

개요: CRIS에서 logical resource 사용하는 방법

Step 1: Logical resource를 사용한 스펙 디자인하기

Step 2: Logical resource의 정의 작성하기

Step 3: Physical state와의 관계 정의하기 (invariant)

Step 4: Logical resource를 이용하여 실제로 증명하기

예제

1. 간단한 메모리 모듈

~~2. 기능이 추가된
메모리 모듈~~

~~3. 자료구조 (스택)~~

기존의 메모리 추상화 방법 이해하기 (Hoare Logic)

$$\frac{\ell \in \text{dom}(M)}{\langle x := \text{alloc}(), \Sigma, M \rangle \Rightarrow \langle \text{skip}, \Sigma[x := \ell], M[\ell := \text{skull}] \rangle}$$

할당: 새로운 ℓ 을 찾아서 맵 만들기

$$\frac{\Sigma[x] = \ell \quad M[\ell] = v}{\langle y := \text{load}(x), \Sigma, M \rangle \Rightarrow \langle \text{skip}, \Sigma[y := v], M \rangle}$$

읽기: ℓ 의 값을 M 으로부터 얻기

$$\frac{\Sigma[x] = \ell \quad \ell \in \text{dom}(M)}{\langle \text{store}(x, w), \Sigma, M \rangle \Rightarrow \langle \text{skip}, \Sigma, M[\ell := w] \rangle}$$

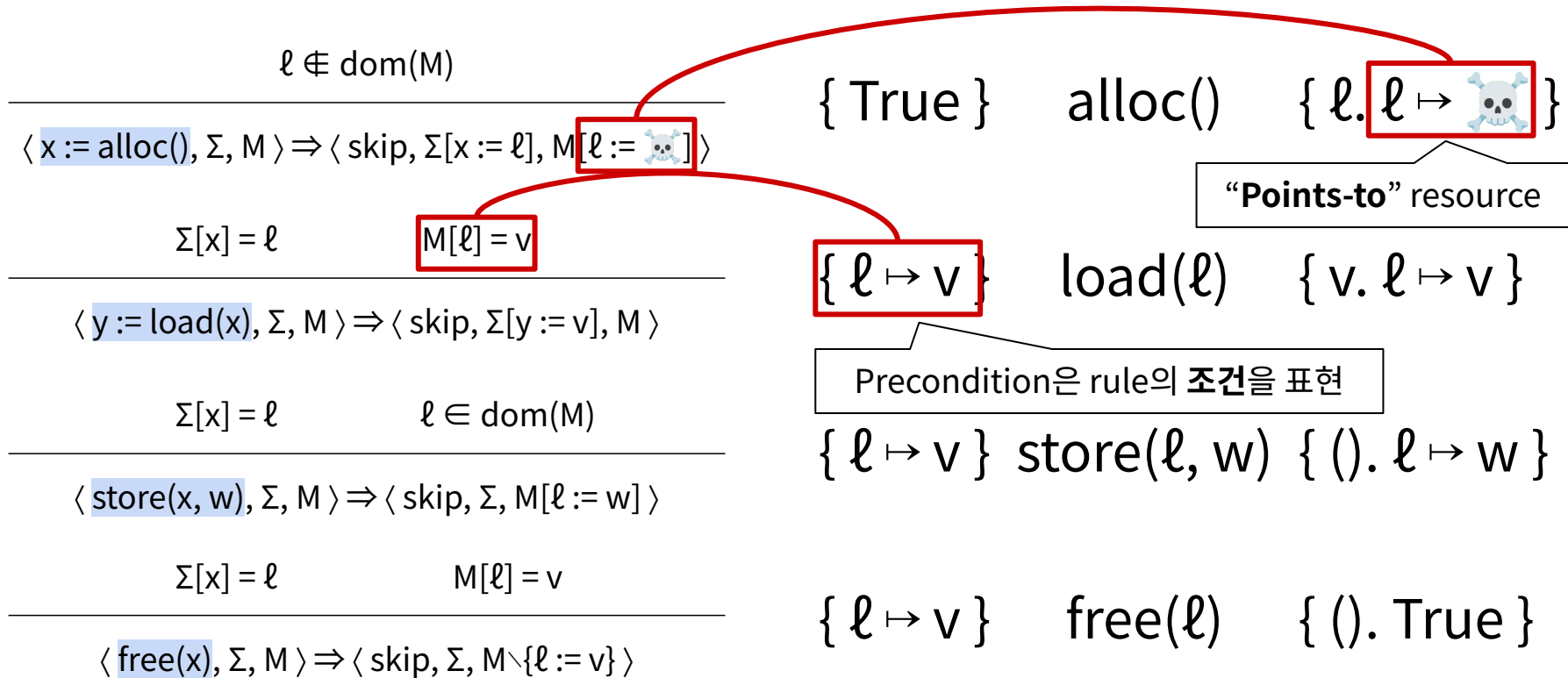
쓰기: ℓ 의 새 값을 M 에 쓰기

$$\frac{\Sigma[x] = \ell \quad M[\ell] = v}{\langle \text{free}(x), \Sigma, M \rangle \Rightarrow \langle \text{skip}, \Sigma, M \setminus \{\ell := v\} \rangle}$$

해제: ℓ 의 정보를 M 에서 지우기

$M \in \text{Loc} \rightarrow \text{Val}$
 ntext
 ry state

기존의 메모리 추상화 방법 이해하기 (Hoare Logic)



같은 아이디어를 CRIS에서 따라하기 — 모델 구현

$\ell \in \text{dom}(M)$

$\langle x := \text{alloc}(), \Sigma, M \rangle \Rightarrow \langle \text{skip}, \Sigma[x := \ell], M[\ell := \text{skull}] \rangle$

def alloc() : itree $\equiv$

$M \leftarrow \text{cgetU}();$

$\ell \leftarrow \text{choose}(\text{Loc} \setminus \text{dom}(M));$

$\text{cputU}(M[\ell := \text{skull}]);$

Ret ℓ

$\Sigma[x] = \ell$

$M[\ell] = v$

$\langle y := \text{load}(x), \Sigma, M \rangle \Rightarrow \langle \text{skip}, \Sigma[y := v], M \rangle$

def load(ℓ) : itree $\equiv$

$M \leftarrow \text{cgetU}();$

Ret $M[\ell]$

[모듈-로컬 상태 접근하기]

cgetU: 상태 읽기

cputU: 상태 쓰기

같은 아이디어를 CRIS에서 따라하기 — 모델 스펙

$$\{ \text{True} \} \quad \text{alloc}() \quad \{ \ell. \ell \mapsto \text{skull} \} \qquad \{ \ell \mapsto v \} \quad \text{load}(\ell) \quad \{ v. \ell \mapsto v \}$$

def alloc() : itree $\equiv$

$\ell \leftarrow$ choose(Loc);

Guarantee($\ell \mapsto \text{skull}$);

Ret ℓ

“너가 v 와 $\ell \mapsto v$ 를 주면
내가 $\ell \mapsto v$ 를 다시 보장해 줄게”

def load(ℓ) : itree $\equiv$

$v \leftarrow$ take(Val);

Assume($\ell \mapsto v$);

Guarantee($\ell \mapsto v$);

Ret v

“내가 ℓ 을 골라서 $\ell \mapsto \text{skull}$ 을 보장해 줄게”

빨간색: Caller의 의무
파란색: Callee의 의무

요약

def alloc() : itree $\equiv$

M $\leftarrow$ cgetU();

$\ell \leftarrow$ choose(Loc \ dom(M));

cputU(M[$\ell :=$ ]);

Ret ℓ

def load(ℓ) : itree $\equiv$

M $\leftarrow$ cgetU();

Ret M[ℓ]

$\sqsubseteq_{\text{beh}}$

def alloc() : itree $\equiv$

$\ell \leftarrow$ choose(Loc);

Guarantee($\ell \mapsto$ );

Ret ℓ

def load(ℓ) : itree $\equiv$

v $\leftarrow$ take(Val);

Assume($\ell \mapsto$ v);

Guarantee($\ell \mapsto$ v);

Ret v

Next Step

Step 2: $\ell \mapsto v$ 정의하기

Step 3: M과 관계 짓기

정의 작성하는 데 유용한 Standard Library

- 희소식: Logical resource 정의할 때 사실 resource algebra를 몰라도 된다!
 - 직접 resource algebra를 정의해서 쓰면 강력하지만 때로는 쓸데없이 복잡하다.
(Algebra의 성질들 정의/증명 필요: valid condition ($\checkmark$), update condition ($\sim\sim>$), 등등)
- Iris (Jung et al. POPL'15) 에 이미 충분한 **high-level library**가 구현되어 있다.
⇒ 필요한 것들 조합해서 쓰기만 하기

예시

- Logical key-value storage
- Monotone한 값에 대해 과거 snapshot 기억하기
- (기타 등등)

Standard Library: Logical key-value storage

중앙 정보

$\text{GMap } M : \text{iProp } \Sigma$

$M = \{ k_1 := v_1, k_2 := v_2, \dots, k_n := v_n \}$

내부 invariant에서 사용하는 모든 정보
(예시: Logical kv storage가 physical memory state와 같다는 invariant)

각 key별 local한 정보

$k_1 \hookrightarrow v_1 : \text{iProp } \Sigma$

$k_2 \hookrightarrow v_2 : \text{iProp } \Sigma$

...

$k_n \hookrightarrow v_n : \text{iProp } \Sigma$

유저한테 노출하는 작은 정보
(예시: $\ell \mapsto v \triangleq \ell \hookrightarrow v$)

Standard Library: Logical key-value storage

(GMap-Lookup)

Separating Conjunction ($P * Q$):
 P 도 갖고 있고 Q 도 갖고 있고 ($\wedge$ 와 유사)

$$\text{GMap } M * k \hookrightarrow v \vdash^{\Gamma} M[k] = v^{\neg}$$

(GMap-Insert)

 $k \notin \text{dom}(M)$

Update Modality ($| \Rightarrow P$):
 리소스 업데이트를 통해 P 를 얻을 수 있다.

$$\text{GMap } M \vdash | \Rightarrow \text{GMap } (M[k:=v]) * k \hookrightarrow v$$

(GMap-Update)

$$\text{GMap } M * k \hookrightarrow v \vdash | \Rightarrow \text{GMap } (M[k:=v']) * k \hookrightarrow v'$$

실제 invariant 작성하기

- CRIS 에서 모듈별 invariant는 **lst** 로 정의 가능

lst : `src_state_type` $\rightarrow$ `tgt_state_type` $\rightarrow$ `iProp` Σ

```
def alloc() : itree  $\equiv$ 
   $\ell \leftarrow$  choose(Loc);
  Guarantee( $\ell \mapsto \text{skull}$ );
  Ret  $\ell$ 
```

스펙에서는 logical resource 사용
 $\Rightarrow$ physical state 가 없다

```
def alloc() : itree  $\equiv$ 
  M  $\leftarrow$  cgetU();
   $\ell \leftarrow$  choose(Loc  $\setminus$  dom(M));
  cputU(M[ $\ell := \text{skull}$ ]);
  Ret  $\ell$ 
```


실제 invariant 작성하기

- CRIS 에서 모듈별 invariant는 **lst** 로 정의 가능

$$\mathbf{lst} : \text{src_state_type} \rightarrow \text{tgt_state_type} \rightarrow \text{iProp } \Sigma$$

$$\text{lst} = \lambda \text{ st_src st_tgt,}$$

$$\exists M. \text{GMap } M * \lceil \text{st_tgt} = M \uparrow \rceil$$

Logical resource는 실제 physical state와 같은 정보를 들고 있다

⇒ 스펙을 쓰는 유저는 $\ell \mapsto v$ ($\triangleq \ell \hookrightarrow v$) 만 가지는데,
이것의 의미는 invariant에서 physical memory state와 연결됨

Alloc 예제 증명

def alloc() : itree $\equiv$

 $M \leftarrow \text{cgetU}();$
 $\ell \leftarrow \text{choose}(\text{Loc} \setminus \text{dom}(M));$
 $\text{cputU}(M[\ell := \text{skull}]);$
Ret ℓ

$\sqsubseteq_{\text{beh}}$

def alloc() : itree $\equiv$

$\ell \leftarrow \text{choose}(\text{Loc});$
 $\text{Guarantee}(\ell \mapsto \text{skull});$
Ret ℓ

----- 증명자가 가지고 있는 정보 / 리소스 -----

$\text{GMap } M * \ulcorner \text{st_tgt} = M \uparrow \urcorner$

해설

초기엔 lst 하나만
가지고 시작

Alloc 예제 증명

def alloc() : itree $\equiv$

$M \leftarrow \text{cgetU}();$

 $\ell \leftarrow \text{choose}(\text{Loc} \setminus \text{dom}(M));$

$\text{cputU}(M[\ell := \text{skull}]);$

Ret ℓ

$\sqsubseteq_{\text{beh}}$

def alloc() : itree $\equiv$

 $\ell \leftarrow \text{choose}(\text{Loc});$

$\text{Guarantee}(\ell \mapsto \text{skull});$

Ret ℓ

----- 증명자가 가지고 있는 정보 / 리소스 -----

$\text{GMap } M * \ulcorner \text{st_tgt} = M \uparrow \urcorner$

$\ell \in \text{Loc} \setminus \text{dom}(M)$

해설

구현 side: ℓ 을 하나 줌

스펙 side: 같은 ℓ 을 보장해 줌

Alloc 예제 증명

def alloc() : itree $\equiv$

$M \leftarrow \text{cgetU}();$

$\Rightarrow \ell \leftarrow \text{choose}(\text{Loc} \setminus \text{dom}(M));$

$\text{cputU}(M[\ell := \text{skull}]);$

Ret ℓ

$\sqsubseteq_{\text{beh}}$

def alloc() : itree $\equiv$

$\Rightarrow \ell \leftarrow \text{choose}(\text{Loc});$

$\text{Guarantee}(\ell \mapsto \text{skull});$

Ret ℓ

----- 증명자가 가지고 있는 정보 / 리소스 -----

$\text{GMap}(M[\ell := \text{skull}]) * \ulcorner \text{st_tgt} = M \uparrow \urcorner$

$\ell \in \text{Loc} \setminus \text{dom}(M)$

$\ell \mapsto \text{skull}$

해설

Logical resource를
업데이트 해서 points-to
리소스를 만들기

Alloc 예제 증명

def alloc() : itree $\equiv$

$M \leftarrow \text{cgetU}();$

$\rightarrow \ell \leftarrow \text{choose}(\text{Loc} \setminus \text{dom}(M));$

$\text{cputU}(M[\ell := \text{skull}]);$

Ret ℓ

$\sqsubseteq_{\text{beh}}$

def alloc() : itree $\equiv$

$\ell \leftarrow \text{choose}(\text{Loc});$

$\rightarrow \text{Guarantee}(\ell \mapsto \text{skull});$

Ret ℓ

----- 증명자가 가지고 있는 정보 / 리소스 -----

$\text{GMap } (M[\ell := \text{skull}]) * \ulcorner \text{st_tgt} = M \uparrow \urcorner$

$\ell \in \text{Loc} \setminus \text{dom}(M)$

$\ell \mapsto \text{skull}$

해설

$\ell \mapsto \text{skull}$ 를 반납하며
Guarantee 실행

Alloc 예제 증명

def alloc() : itree $\equiv$

$M \leftarrow \text{cgetU}();$

$\ell \leftarrow \text{choose}(\text{Loc} \setminus \text{dom}(M));$

 $\text{cputU}(M[\ell := \text{skull}]);$

Ret ℓ

$\sqsubseteq_{\text{beh}}$



def alloc() : itree $\equiv$

$\ell \leftarrow \text{choose}(\text{Loc});$

Guarantee ($\ell \mapsto \text{skull}$);

Ret ℓ

----- 증명자가 가지고 있는 정보 / 리소스 -----

$\text{GMap } (M[\ell := \text{skull}]) *$
 $\ulcorner \text{st_tgt}' = (M[\ell := \text{skull}]) \uparrow \urcorner$

$\ell \in \text{Loc} \setminus \text{dom}(M)$

Ist

해설

cputU 를 통해 physical
memory state를
업데이트하면 Ist 다시 성립

Alloc 예제 증명

def alloc() : itree $\equiv$

$M \leftarrow \text{cgetU}();$

$\ell \leftarrow \text{choose}(\text{Loc} \setminus \text{dom}(M));$

$\text{cputU}(M[\ell := \text{skull}]);$

 **Ret** ℓ

$\sqsubseteq_{\text{beh}}$



def alloc() : itree $\equiv$

$\ell \leftarrow \text{choose}(\text{Loc});$

$\text{Guarantee}(\ell \mapsto \text{skull});$

Ret ℓ

----- 증명자가 가지고 있는 정보 / 리소스 -----

$\text{GMap } (M[\ell := \text{skull}]) * \text{「 st_tgt' = } (M[\ell := \text{skull}]) \uparrow \text{」}$

$\ell \in \text{Loc} \setminus \text{dom}(M)$

Ist

해설

양쪽 모두 같은 값을
return하며 Ist가
성립하므로 증명 종료

Load 예제 증명

def load(ℓ) : itree $\equiv$
 $M \leftarrow \text{cgetU}();$
Ret $M[\ell]$

$\sqsubseteq_{\text{beh}}$

def load(ℓ) : itree $\equiv$
 $v \leftarrow \text{take}(\text{Val});$
 $\text{Assume}(\ell \mapsto v);$
 $\text{Guarantee}(\ell \mapsto v);$
Ret v

----- 증명자가 가지고 있는 정보 / 리소스 -----

$\text{GMap } M * \ulcorner \text{st_tgt} = M \uparrow \urcorner$

해설

초기엔 lst 하나만
가지고 시작

Load 예제 증명

def load(ℓ) : itree $\equiv$
 $M \leftarrow \text{cgetU}();$
Ret $M[\ell]$

$\sqsubseteq_{\text{beh}}$

def load(ℓ) : itree $\equiv$
 $v \leftarrow \text{take}(\text{Val});$
 **Assume**($\ell \mapsto v$);
Guarantee($\ell \mapsto v$);
Ret v

----- 증명자가 가지고 있는 정보 / 리소스 -----

$\text{GMap } M * \ulcorner \text{st_tgt} = M \uparrow \urcorner$

$\ell \mapsto v$

해설

스펙 side에서
 $\text{take}/\text{Assume}$ 하면 이러한
리소스를 가정에 추가

Load 예제 증명

def load(ℓ) : itree $\equiv$
 $\rightarrow$ M $\leftarrow$ cgetU();
Ret M[ℓ]

$\sqsubseteq_{\text{beh}}$

def load(ℓ) : itree $\equiv$
 v $\leftarrow$ take(Val);
Assume($\ell \mapsto v$);
Guarantee($\ell \mapsto v$);
Ret v

----- 증명자가 가지고 있는 정보 / 리소스 -----

GMap M * $\ulcorner$ st_tgt = M $\uparrow$ $\urcorner$
 $\ell \mapsto v$ M[ℓ] = v

해설

GMap-Lookup 렘마를 통해
 M[ℓ] = v 라는 결론 도출

Load 예제 증명

def load(ℓ) : itree $\equiv$
 $\rightarrow$ M $\leftarrow$ cgetU();
Ret M[ℓ]

$\sqsubseteq_{\text{beh}}$ **def** load(ℓ) : itree $\equiv$
 v $\leftarrow$ take(Val);
 Assume($\ell \mapsto v$);
 Guarantee($\ell \mapsto v$);
Ret v

----- 증명자가 가지고 있는 정보 / 리소스 -----

GMap M * $\ulcorner$ st_tgt = M $\uparrow$ $\urcorner$

$\ell \mapsto v$

M[ℓ] = v

해설

$\ell \mapsto v$ 를 그대로 반납하며
 Guarantee 실행

Load 예제 증명

```
def load( $\ell$ ) : itree  $\equiv$ 
  M  $\leftarrow$  cgetU();
   Ret M[ $\ell$ ]
```

$\sqsubseteq_{\text{beh}}$

```
def load( $\ell$ ) : itree  $\equiv$ 
  v  $\leftarrow$  take(Val);
  Assume( $\ell \mapsto v$ );
  Guarantee( $\ell \mapsto v$ );
   Ret v
```

----- 증명자가 가지고 있는 정보 / 리소스 -----

$\text{GMap } M * \ulcorner \text{st_tgt} = M \uparrow \urcorner$

 lst

$M[\ell] = v$

해설

같은 return값 보장 & lst 성립
 $\Rightarrow$ Load 증명 종료